

# AForce OS — Source-Code Excerpts

---

## AForce OS — Source-Code Excerpts for Cited Functions

---

**Companion to:** `aforce-os-engineering-brief.md` **Audience:** Outside counsel and any technical reviewer the firm engages **Date:** April 29, 2026

This document reproduces the **actual source code** for every function the engineering brief cites. Code is verbatim; line numbers are the same as in the live repository as of the date above.

The excerpts are organized to mirror the brief's section order: §2 Scoring Engine, §3 Deterministic AI Layer, §4 Autopilot, §5 Autoscan. Each excerpt is preceded by a one-line description and the file path + line range it was pulled from.

---

### Section 2 — Scoring Engine

#### 2.6 Score-band classification

`artifacts/aforce-os/utils/scoringEngine.ts:38–43`

```
function resolveState(score: number): PerformanceLevel {  
  if (score >= 90) return 'PEAK';  
  if (score >= 75) return 'BALANCED';  
  if (score >= 60) return 'RECOVERING';  
  return 'DEPLETED';  
}
```

#### 2.5 Continuous decay (per-minute rate)

`artifacts/aforce-os/utils/scoringEngine.ts:166–202`

```
function computeDecayPerMinute(state: UserState): number {  
  const weight = Math.max(60, state.bodyWeightLbs || 150);  
  const activity = Math.max(0, state.activityLevel || 0);  
  // Prefer real OpenWeather data when the api-server has it; fall back  
  // to the heatLoad-derived approximation so the score still renders  
  // before the first weather lookup completes (or when offline).  
  const tempC = state.weatherTempC != null  
    ? state.weatherTempC  
    : 20 + (state.heatLoad ?? 0) * 1.2; // ~20°C @ 0 → 32°C @ 10  
  const humidity = state.weatherHumidity != null ? state.weatherHumidity : 50;  
  
  const baseDecay = 0.4 * (weight / 150) + 0.1 * activity;  
  const heatFactor = Math.max(0, (tempC - 25) * 0.3);  
  const humidityFactor = Math.max(0, ((humidity - 50) / 10) * 0.2);  
  
  let perMin = baseDecay + heatFactor + humidityFactor;  
  // Sleep mode halves decay per spec.  
  if (!state.isAwake) perMin *= 0.5;  
  // Clutch mode multiplier (T3): ×1.3 while clutch_access_enabled is on.  
  if (state.clutchActive) perMin *= 1.3;  
  if (state.socialMode?.active) {  
    perMin *= activeDecayMultiplier(state.socialMode.drinks);  
  }  
  return perMin;  
}
```

#### 2.5 Continuous decay (integrated over time)

artifacts/aforce-os/utills/scoringEngine.ts:215-231

```
function computeDecayPoints(state: UserState, minutesSinceLast: number): number {
  const baseline = computeDecayPerMinute(state) * Math.max(0, minutesSinceLast);

  let boost = 0;
  if (state.clutchDecayBoostUntil) {
    const boostEndMs = state.clutchDecayBoostUntil.getTime();
    const boostStartMs = boostEndMs - 10 * 60 * 1000;
    const intakeMs = state.lastIntakeTime.getTime();
    const nowMs = Date.now();
    const overlapStart = Math.max(boostStartMs, intakeMs);
    const overlapEnd = Math.min(boostEndMs, nowMs);
    if (overlapEnd > overlapStart) {
      boost = 0.5 * ((overlapEnd - overlapStart) / 60000);
    }
  }
  return baseline + boost;
}
```

## 2.3 Score formula (sum of bounded terms)

artifacts/aforce-os/utills/scoringEngine.ts:363-368

```
const raw = baseIntake + aforceBonus + recency + consistency + context + recoveryMomentum
  + symptomPenalty + urinePenalty + outputStress + sleepCarry
  + recovery.delta + confirmation;

return Math.max(0, Math.min(100, Math.round(raw)));
```

## 2.4 Per-event absorption — header comment (rubric)

artifacts/aforce-os/services/hydrationScoreService.ts:1-34

```
/**
 * Hydration Scoring Engine — per-event, time-windowed.
 *
 * STATUS: Product design rubric (AForce IP), not a clinical model.
 * 1 unit = 12 oz
 *
 * Water: +0.5 pts/oz (so 12oz=+6, 16oz=+8, 24oz=+12, 32oz=+16)
 * AForce:
 *   Berry Blast +10
 *   Watermelon Surge +10 (+2 if Heat Guard active)
 *   Soursop Edge +11 (+2 if scoreBefore < 40)
 *
 * Absorption cap: ≤1.5 units per rolling 20-min window. Excess
 * intake is absorbed at 75% efficiency.
 *
 * Absorption curve:
 *   Water: 60% immediate, 40% over 12.5 min
 *   AForce: 70% immediate, 30% over 25 min
 */
```

## 2.4 Per-event absorption — exported constants

artifacts/aforce-os/services/hydrationScoreService.ts:40-63

```

export const HYDRATION_UNIT_OZ = 12;
export const WATER_PTS_PER_OZ = 0.5;

export const ABSORPTION_CAP_UNITS = 1.5;
export const ABSORPTION_WINDOW_MIN = 20;
export const EXCESS_EFFICIENCY = 0.75;

export const WATER_IMMEDIATE_PCT = 0.6;
export const WATER_DELAYED_DURATION_MIN = 12.5;

export const AFORCE_IMMEDIATE_PCT = 0.7;
export const AFORCE_DELAYED_DURATION_MIN = 25;

export const AFORCE_BASE_IMPACT: Record<ProductFlavor, number> = {
  berry: 10,
  watermelon: 10,
  soursop: 11,
  unflavored: 10,
};

```

## Section 3 — Deterministic AI Layer

### 3.2 / 3.3 Pipeline-order: Social Mode preempts the standard ladder

artifacts/aforce-os/utlils/scoringEngine.ts:537-591

```

function generateCommand(level: PerformanceLevel, state: UserState, score: number, social: ScoreEngineOutput['social']):

  // Social Mode takes precedence over the standard PEAK/BALANCED/etc
  // protocol — the user is actively drinking (or just stopped) and the
  // coach must speak to that, not generic hydration math.
  if (social) {
    const social_cmd = generateSocialCommand(state, social);
    if (social_cmd) return social_cmd;
  }

  // Sleep mode: morning command if overnight deficit is significant
  if (state.overnightLossOz > 8 && !state.hasSeenMorningCommand) {
    const oz = Math.max(16, Math.round(state.overnightLossOz));
    return {
      id: 'cmd-morning',
      action: i18n.t('coach.morning_action', { oz }),
      explanation: i18n.t('coach.morning_explanation', { oz: state.overnightLossOz }),
      urgencyLevel: 'high',
      estimatedImpact: '+12 to score',
    };
  }

  switch (level) {
    case 'PEAK':
      return { id: 'cmd-peak', action: i18n.t('coach.peak_action', { score }),
        explanation: i18n.t('coach.peak_explanation'),
        urgencyLevel: 'low', estimatedImpact: '+2 to score' };
    case 'BALANCED':
      return { id: 'cmd-balanced', action: i18n.t('coach.balanced_action', { score }),
        explanation: i18n.t('coach.balanced_explanation'),
        urgencyLevel: 'medium', estimatedImpact: '+5 to score' };
    case 'RECOVERING':
      return { id: 'cmd-recovering', action: i18n.t('coach.recovering_action', { score }),
        explanation: i18n.t('coach.recovering_explanation'),
        urgencyLevel: 'high', estimatedImpact: '+10 to score' };
    case 'DEPLETED':
      return { id: 'cmd-depleted', action: i18n.t('coach.depleted_action', { score }),
        explanation: i18n.t('coach.depleted_explanation'),
        urgencyLevel: 'critical', estimatedImpact: '+18 to score' };
  }
}

```

### 3.3 Social Mode command generator (safety-class branch)

```

function generateSocialCommand(state: UserState, social: NonNullable<ScoreEngineOutput['social']>): Command | null {
  // Recovery Mode (drinking ended within 8h)
  if (social.inRecoveryWindow && !social.active) {
    return {
      id: 'cmd-social-recovery',
      action: i18n.t('coach.social_recovery_action'),
      explanation: i18n.t('coach.social_recovery_explanation'),
      urgencyLevel: 'high',
      estimatedImpact: '+15 to score',
    };
  }
  if (!social.active) return null;
  const drinks = state.socialMode?.drinks ?? [];
  const lastDrink = drinks.length > 0 ? drinks[drinks.length - 1] : null;
  const minutesSinceDrink = lastDrink
    ? (Date.now() - lastDrink.loggedAt.getTime()) / 60000
    : Infinity;

  // CRITICAL impairment → strongest, most protective copy.
  if (social.impairment.level === 'CRITICAL') {
    return {
      id: 'cmd-social-stop-critical',
      action: i18n.t('coach.social_stop_action'),
      explanation: i18n.t('coach.social_do_not_drive_explanation'),
      urgencyLevel: 'critical',
      estimatedImpact: '+18 to score',
    };
  }
  // HIGH impairment → "do not drive" + transportation prompt.
  if (social.impairment.level === 'HIGH') {
    return {
      id: 'cmd-social-do-not-drive',
      action: i18n.t('coach.social_do_not_drive_action'),
      explanation: i18n.t('coach.social_do_not_drive_explanation'),
      urgencyLevel: 'critical',
      estimatedImpact: '+15 to score',
    };
  }
  // Just logged a drink → hydration command (within 5 min).
  if (minutesSinceDrink <= 5 && lastDrink?.hydrated !== true) {
    return {
      id: 'cmd-social-hydrate',
      action: i18n.t('coach.social_drink_water_action'),
      explanation: i18n.t('coach.social_drink_water_explanation'),
      urgencyLevel: 'high',
      estimatedImpact: '+8 to score',
    };
  }
  // CRITICAL/HIGH hangover risk → push AForce RTD harder.
  if (social.hangoverRisk.level === 'CRITICAL' || social.hangoverRisk.level === 'HIGH') {
    return {
      id: 'cmd-social-rtd',
      action: i18n.t('coach.social_take_rtd_action'),
      explanation: i18n.t('coach.social_take_rtd_explanation', { score: social.hangoverRisk.score }),
      urgencyLevel: 'critical',
      estimatedImpact: '+12 to score',
    };
  }
  return {
    id: 'cmd-social-pace',
    action: i18n.t('coach.social_slow_intake_action'),
    explanation: i18n.t('coach.social_slow_intake_explanation', { count: drinks.length }),
    urgencyLevel: 'medium',
    estimatedImpact: '+5 to score',
  };
}

```

### 3.3 BAC estimator (Widmark approximation)

```

export function estimateBAC(inputs: BACInputs): BACEstimate {
  const drinks = inputs.drinks ?? [];
  const now = inputs.now ?? Date.now();
  const weightLbs = Math.max(80, inputs.bodyWeightLbs ?? 170);
  const r = widmarkR(inputs.sex);
  const bodyMassG = weightLbs * LBS_TO_G;
  const ateRecently = inputs.ateRecently === true;

  const current = bacAt(now, drinks, bodyMassG, r, ateRecently);
  const past = bacAt(now - 15 * 60 * 1000, drinks, bodyMassG, r, ateRecently);

  // ±0.01 widening — Widmark is roughly ±15-20% in the literature.
  const rangeLow = Math.max(0, Math.round((current.bac - 0.01) * 1000) / 1000);
  const rangeHigh = Math.max(rangeLow, Math.round((current.bac + 0.01) * 1000) / 1000);

  let trend: BACEstimate['trend'];
  const delta = current.bac - past.bac;
  if (Math.abs(delta) < 0.005) trend = 'steady';
  else if (delta > 0) trend = 'rising';
  else trend = 'falling';

  const overshoot = Math.max(0, current.bac - CLEAR_THRESHOLD);
  const minutesToClear = (overshoot / ELIMINATION_PER_HOUR) * 60;
  const timeToClearMinutes = Math.ceil(minutesToClear / 5) * 5;

  // ... confidence + notes ...

  return { rangeLow, rangeHigh, trend, confidence, timeToClearMinutes, notes };
}

```

### 3.4 Guardian Risk Engine (composite risk)

artifacts/aforce-os/utills/scoringEngine.ts:728-755

```

export function guardianRiskScore(input: {
  hydrationPercent: number;
  bodyWeightLbs: number;
  activeMinutes: number;
  heatIndex: number;
  sweatRate: number;
  coreTempEstimate: number;
  quarter: number;
  pH: number;
}): number {
  const dehydrationContribution = (100 - input.hydrationPercent) * 0.45;
  const heatContribution = Math.min(30, Math.max(0, (input.heatIndex - 75) * 0.8));
  const exertionContribution = Math.min(20, input.activeMinutes * 0.18);
  const coreTempContribution = Math.min(20, Math.max(0, (input.coreTempEstimate - 99) * 12));
  const phContribution = Math.min(10, Math.max(0, (7.0 - input.pH) * 6));
  const quarterContribution = input.quarter > 2 ? 4 : 0;
  return Math.max(0, Math.min(100, Math.round(
    dehydrationContribution + heatContribution + exertionContribution +
    coreTempContribution + phContribution + quarterContribution
  )));
}

export function guardianTier(score: number): 'OPTIMAL' | 'WATCH' | 'MODERATE' | 'CRITICAL' {
  if (score >= 75) return 'CRITICAL';
  if (score >= 50) return 'MODERATE';
  if (score >= 25) return 'WATCH';
  return 'OPTIMAL';
}

```

### 3.4 Clutch Strategy (per-player coach recommendation)

artifacts/aforce-os/utills/scoringEngine.ts:790-838

```

export function clutchRecommendation(input: {
  hydrationScore: number;
  position?: string;
}): ClutchRecommendation {
  const tier = clutchTier(input.hydrationScore);
  switch (tier) {
    case 'PLATINUM':
      return { tier, action: 'maintain', fluidOz: 8, sticks: 0, recheckMinutes: 30,
        command: 'Maintain. 8 oz water at next break.',
        detail: 'Keep rotation. Recheck end of quarter.' };
    case 'STABLE':
      return { tier, action: 'top_off', fluidOz: 12, sticks: 1, recheckMinutes: 20,
        command: '12 oz + 1 stick at next dead ball.',
        detail: 'Hold rotation. Recheck in 20 min.' };
    case 'RECOVERY':
      return { tier, action: 'restore', fluidOz: 16, sticks: 2, recheckMinutes: 10,
        command: '16 oz + 2 sticks now.',
        detail: 'Move to shaded area. Recheck in 10 min.' };
    case 'DEPLETED':
    default:
      return { tier, action: 'pull', fluidOz: 24, sticks: 3, recheckMinutes: 5,
        command: 'PULL FROM ROTATION. 24 oz + 3 sticks.',
        detail: 'Cooling protocol. Recheck core temp in 5 min.' };
  }
}

```

## 2.6 Default recheck cadence by band (overridden by Autopilot when active)

artifacts/aforce-os/utils/scoringEngine.ts:441-448

```

function getBaseRiskMinutes(level: PerformanceLevel, minutesSinceLast: number): number {
  const remaining = Math.max(0, 60 - minutesSinceLast);
  switch (level) {
    case 'PEAK': return Math.max(20, remaining);
    case 'BALANCED': return Math.max(15, Math.floor(remaining * 0.7));
    case 'RECOVERING': return Math.max(10, Math.floor(remaining * 0.4));
    case 'DEPLETED': return Math.max(5, Math.floor(remaining * 0.2) + 1);
  }
}

```

## Section 4 — Autopilot

### 4.3 The autopilot cadence ladder (the central function)

artifacts/aforce-os/services/sweatRateEngine.ts:481-493

```

/**
 * Autopilot recheck cadence — driven entirely by deficit %.
 * Spec:
 * ≥ 4% → 8 min / critical
 * ≥ 2% → 12 min / high
 * else → 20 min / moderate
 * Recovery window is fixed at 4 hours (spec §4).
 */
export function deriveAutopilot(deficitPct: number): SweatAutopilot {
  if (deficitPct >= 4) return { intervalMin: 8, urgency: 'critical', recoveryWindowHours: 4 };
  if (deficitPct >= 2) return { intervalMin: 12, urgency: 'high', recoveryWindowHours: 4 };
  return { intervalMin: 20, urgency: 'moderate', recoveryWindowHours: 4 };
}

```

### 4.4 Reducer wiring — SET\_SWEAT\_AUTOPILOT (atomic 3-field transition)

artifacts/aforce-os/store/appStoreReducer.ts:13-35

```

export function reducer(state: AppState, action: Action): AppState {
  switch (action.type) {
    case 'SET_SWEAT_AUTOPILOT': {
      const { autopilot, setAt } = action.payload;
      // When a sweat-driven autopilot window opens, also reset the
      // active recheck countdown to its cadence so the existing
      // RiskTimerDisplay (driven by `timerSeconds`) reflects the
      // autopilot interval instead of the stale engine.riskTimer value.
      const nextTimerSeconds = autopilot
        ? autopilot.intervalMin * 60
        : state.timerSeconds;
      return {
        ...state,
        sweatAutopilot: autopilot,
        sweatAutopilotSetAt: setAt,
        timerSeconds: nextTimerSeconds,
        // A fresh autopilot window invalidates any pending "did you
        // follow the command?" prompt — the new cadence supersedes it.
        pendingConfirmation: autopilot ? false : state.pendingConfirmation,
      };
    }
    // ... other cases ...
  }
}

```

## 4.5 Consumer hook — useHeatGuard returns autopilot when within window

artifacts/aforce-os/hooks/useHeatGuard.ts:102-117

```

// Sweat-driven autopilot wins for the duration of the recovery
// window (spec §4). Outside that window we fall back to a band
-
// derived default so the recheck cadence still reflects current
// physiological strain.
const windowMs = (autopilot?.recoveryWindowHours ?? 0) * HOUR_MS;
const autopilotActive =
  !!autopilot && setAt != null && Date.now() - setAt < windowMs;
const fallback = defaultCadenceForBand(heat.band);

return {
  score: heat.score,
  band: heat.band,
  recheckIntervalMin: autopilotActive ? autopilot!.intervalMin : fallback.intervalMin,
  recheckUrgency: autopilotActive ? autopilot!.urgency : fallback.urgency,
  autopilotActive,
};

```

## 4.7 Sodium-gap audit (the math that closes Hook 5)

artifacts/aforce-os/services/sweatRateEngine.ts:128, 506-511

```

export const AFORCE_SODIUM_PER_UNIT_MG = 25;

// ...

// Spec rules:
// aforce_sodium_total = total_units * 25
// sodium_gap = max(0, sodium_loss - aforce_sodium_total)
const aforceSodiumTotalMg = args.prescription.aforceSticks * AFORCE_SODIUM_PER_UNIT_MG;
const sodiumGapMg = Math.max(0, args.sodiumLossMg - aforceSodiumTotalMg);

```

# Section 5 — Autoscan

## 5.4 Hydration-efficiency formula

artifacts/aforce-os/services/hydrationScanService.ts:41-63

```

/**
 * Per-product hydration efficiency 0..1 per spec:
 * efficiency = M*0.4 + W*0.3 + LS*0.2 - S*0.1
 *
 * M = mineral content    → product.electrolytes / 100
 * W = water content      → product.hydratationSpeed / 100
 * LS = low-sugar quality → 1 - (product.sugar / 100)
 * S = sugar penalty      → product.sugar / 100
 */
export function computeHydrationEfficiency(product: CompareProduct): number {
  const M = Math.max(0, Math.min(1, (product.electrolytes ?? 0) / 100));
  const W = Math.max(0, Math.min(1, (product.hydratationSpeed ?? 0) / 100));
  const sugar01 = Math.max(0, Math.min(1, (product.sugar ?? 0) / 100));
  const LS = 1 - sugar01;
  const S = sugar01;
  const raw = M * 0.4 + W * 0.3 + LS * 0.2 - S * 0.1;
  return Math.max(0, Math.min(1, raw));
}

```

## 5.5 AForce-replacement decision rule

artifacts/aforce-os/services/hydrationScanService.ts:69-101

```

function bestAforceFor(inputs: CompareInputs): CompareResult | undefined {
  const aforce = COMPARE_PRODUCTS.filter((p) => p.isAForce);
  if (aforce.length === 0) return undefined;
  const { results } = computeComparison({ inputs, catalog: aforce });
  return results[0];
}

function buildRecommendation(
  scanned: ScannedProduct,
  inputs: CompareInputs,
  selfFit: CompareResult,
  bestAforce: CompareResult | undefined,
): ScanRecommendation {
  const stateLabel = inputs.state.charAt(0) + inputs.state.slice(1).toLowerCase();
  // CASE 1: scanned product is AForce and already optimal → log it.
  if (scanned.isAForce && selfFit.verdict === 'optimal') {
    return { headline: `${scanned.productName} is optimal for your current state`,
      detail: selfFit.whyItFits,
      command: `Take 1 ${scanned.productName} now with 16 oz water. Recheck in 20 minutes`,
      shouldLog: true };
  }
  // CASE 2: AForce alternative exists and outperforms by > 4 fit-score points.
  if (bestAforce && bestAforce.product.id !== scanned.productId
    && bestAforce.fitScore > selfFit.fitScore + 4) {
    return { headline: `${bestAforce.product.name} is a stronger fit for your ${stateLabel} state`,
      detail: bestAforce.whyItFits,
      aforceEquivalentId: bestAforce.product.id,
      command: `Take 1 ${bestAforce.product.name} now with 16 oz water. Recheck in 20 minutes`,
      shouldLog: false };
  }
  // ... CASES 3 and 4 omitted for brevity, see the file ...
}

```

## Verification

Every excerpt above is a verbatim copy of the file at the cited line range, retrieved on April 29, 2026. Counsel may verify any excerpt by opening the cited file at the cited line range in the repository.

The full source files are available at:



```
artifacts/aforce-os/utills/scoringEngine.ts  
artifacts/aforce-os/services/sweatRateEngine.ts  
artifacts/aforce-os/services/hydrationScoreService.ts  
artifacts/aforce-os/services/hydrationScanService.ts  
artifacts/aforce-os/services/bacEstimationService.ts  
artifacts/aforce-os/store/appStoreReducer.ts  
artifacts/aforce-os/hooks/useHeatGuard.ts
```

Engineering will provide working-code walk-throughs of any excerpt on request.